

YOLOv8n Object Detection

YOLOv8n Object Detection

[Demo Results](#)

[Code Explanation](#)

[Code Structure](#)

[Code Analysis](#)

[Algorithm Overview](#)

[What is YOLO?](#)

Demo Results

In this section, we will learn how to implement YOLOv8-based object detection using the K230.

This section's model supports recognizing nearly 80 types of objects. For the complete list of supported object types, please refer to the `labels` array in the code.

Here is a brief summary:

Transportation:

bicycle, car, motorcycle, airplane, bus, train, truck, boat

Traffic Facilities:

traffic light, fire hydrant, stop sign, parking meter

Animals:

bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe

Personal Items:

backpack, umbrella, handbag, tie, suitcase

Sports Equipment:

frisbee, skis, snowboard, sports ball, kite, baseball bat, baseball glove, skateboard, surfboard, tennis racket

Dining Items:

bottle, wine glass, cup, fork, knife, spoon, bowl

Food:

banana, apple, sandwich, orange, broccoli, carrot, hot dog, pizza, donut, cake

Home Items:

bench, chair, couch, potted plant, bed, dining table, toilet, vase

Electronics:

tv, laptop, mouse, remote, keyboard, cell phone, microwave, oven, toaster, sink, refrigerator, hair drier

Others:

book, clock, scissors, teddy bear, toothbrush

After connecting to the IDE and running the demo code, let's try it out with a few simple scenes.



As you can see, the K230 correctly identifies these objects and provides confidence scores.

This model can recognize a wide variety of objects. I won't list them all here - you can experiment with them based on the table above.

The current demo has been updated with serial output.

For the protocol format, please refer to the [Demo Communication Protocol.xlsx] in the reference materials.

Code Explanation

Code Structure

1. Initialization Phase:

- Load YOLOv8 model
- Load labels
- Set parameters
- Initialize AI2D
- Generate class colors

2. Preprocessing Flow:

- Configure preprocessing
- Calculate padding
- Resize image
- Normalize

3. Detection Flow:

- Run model
- Transpose output

- Postprocess
 - NMS (Non-Maximum Suppression)
4. Drawing Flow:
- Clear display
 - Draw boxes
 - Draw labels
 - Update display
5. Main Loop:
- Get frame
 - Process flow
 - Garbage collection
6. Exit Flow:
- Exit demo
 - Clean up

Code Analysis

For the complete code, please refer to [Source Code Summary / 09.Scene / 03.object_detect_yolov8n]

```
# Custom YOLOv8 detection class
class ObjectDetectionApp(AIBase):
    def
__init__(self, kmodel_path, labels, model_input_size, max_boxes_num, confidence_threshold=0.5, nms
_threshold=0.2, rgb888p_size=[224, 224], display_size=[1920, 1080], debug_mode=0):
    """
    Initialization function
    kmodel_path: Model path
    labels: Label list
    model_input_size: Model input size
    max_boxes_num: Maximum number of detection boxes
    confidence_threshold: Confidence threshold
    nms_threshold: Non-maximum suppression threshold
    rgb888p_size: RGB image size
    display_size: Display size
    debug_mode: Debug mode
    """
    super().__init__(kmodel_path, model_input_size, rgb888p_size, debug_mode)
    self.kmodel_path = kmodel_path
    self.labels = labels
    # Model input resolution
    self.model_input_size = model_input_size
    # Threshold settings
    self.confidence_threshold = confidence_threshold
    self.nms_threshold = nms_threshold
    self.max_boxes_num = max_boxes_num
    # Image resolution from sensor to AI
    self.rgb888p_size = [ALIGN_UP(rgb888p_size[0], 16), rgb888p_size[1]]
```

```

# Display resolution
self.display_size = [ALIGN_UP(display_size[0],16),display_size[1]]
self.debug_mode = debug_mode
# Preset colors for detection boxes
self.color_four = get_colors(len(self.labels))
# Width and height scaling ratios
self.x_factor = float(self.rgb888p_size[0])/self.model_input_size[0]
self.y_factor = float(self.rgb888p_size[1])/self.model_input_size[1]
# Ai2d instance for model preprocessing
self.ai2d = Ai2d(debug_mode)
# Set Ai2d input/output format and type
self.ai2d.set_ai2d_dtype(nn.ai2d_format.NCHW_FMT,nn.ai2d_format.NCHW_FMT,np.uint8,
np.uint8)

# Configure preprocessing operations
def config_preprocess(self,input_image_size=None):
    """
    Configure image preprocessing parameters
    input_image_size: Input image size (optional)
    """
    with ScopedTiming("set preprocess config",self.debug_mode > 0):
        # Initialize ai2d preprocessing configuration
        ai2d_input_size = input_image_size if input_image_size else self.rgb888p_size
        top,bottom,left,right,self.scale =
letterbox_pad_param(self.rgb888p_size,self.model_input_size)
        # Configure padding preprocessing
        self.ai2d.pad([0,0,0,0,top,bottom,left,right], 0, [128,128,128])
        self.ai2d.resize(nn.interp_method.tf_bilinear, nn.interp_mode.half_pixel)
        self.ai2d.build([1,3,ai2d_input_size[1],ai2d_input_size[0]],
[1,3,self.model_input_size[1],self.model_input_size[0]])

# Preprocessing function
def preprocess(self,input_np):
    with ScopedTiming("preprocess",self.debug_mode > 0):
        return [nn.from_numpy(input_np)]

# Postprocessing function
def postprocess(self,results):
    """
    Process model output results
    results: Model output
    """
    with ScopedTiming("postprocess",self.debug_mode > 0):
        new_result = results[0][0].transpose()
        det_res = aidemo.yolov8_det_postprocess(new_result.copy(),
[self.rgb888p_size[1],self.rgb888p_size[0]],
[self.model_input_size[1],self.model_input_size[0]],
[self.display_size[1],self.display_size[0]],
len(self.labels),
self.confidence_threshold,
self.nms_threshold,
self.max_boxes_num)
        return det_res

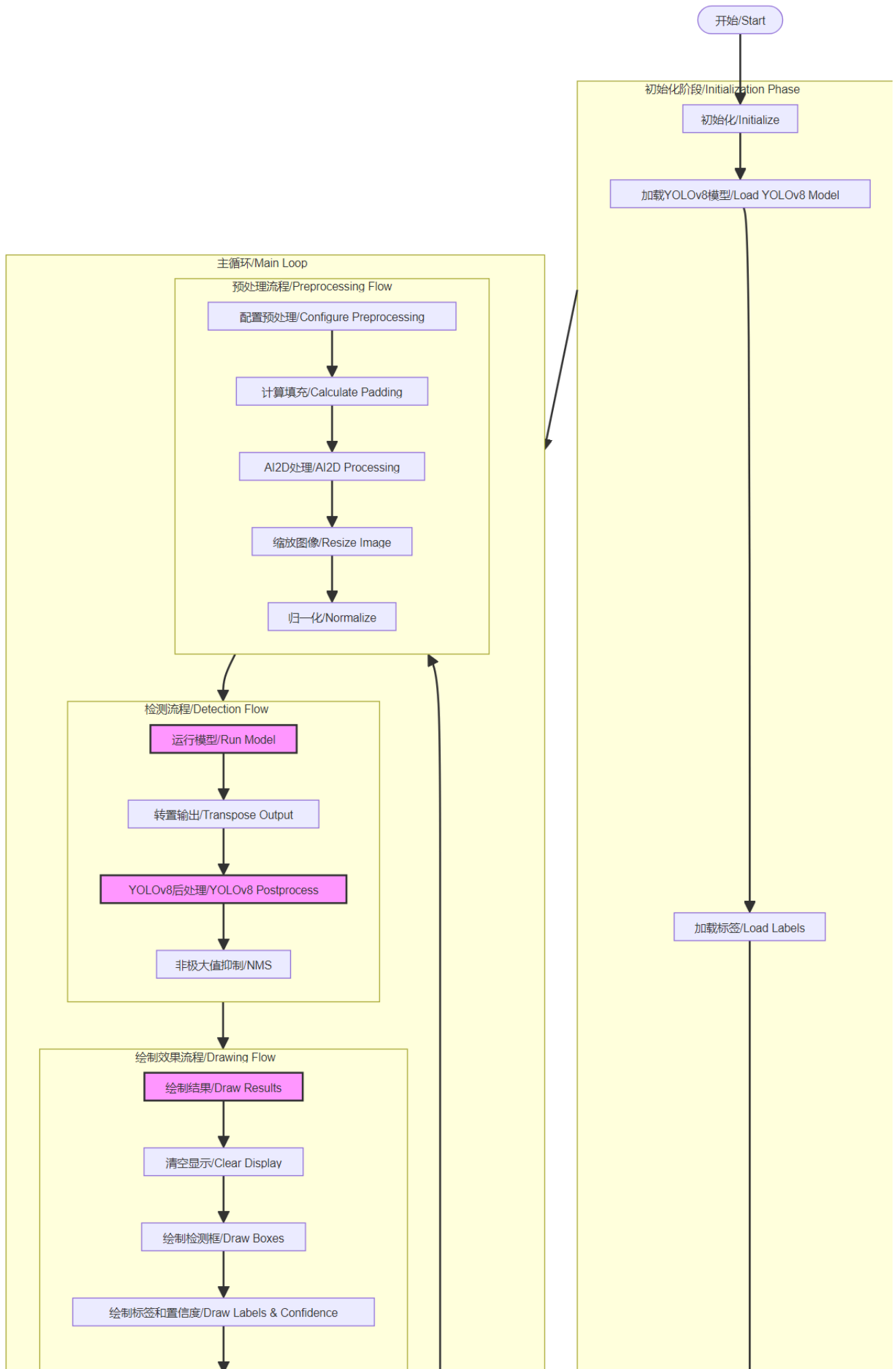
```

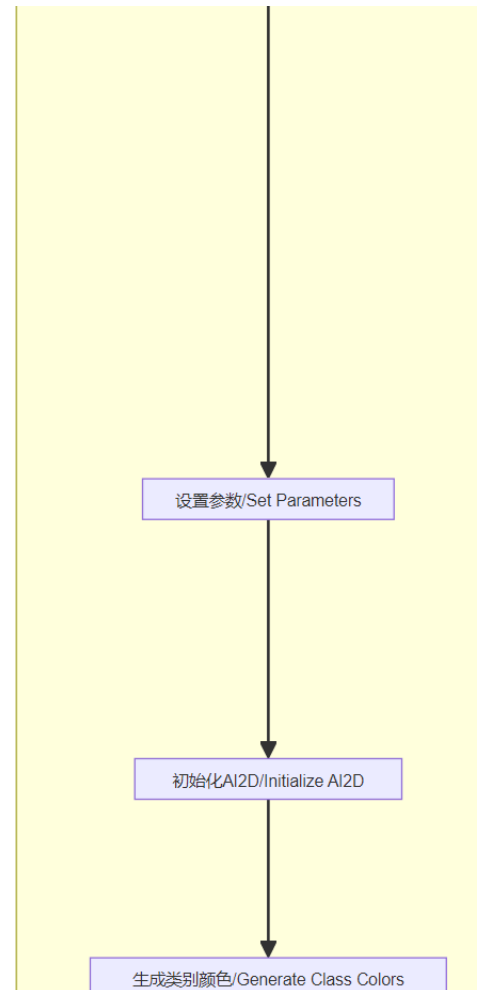
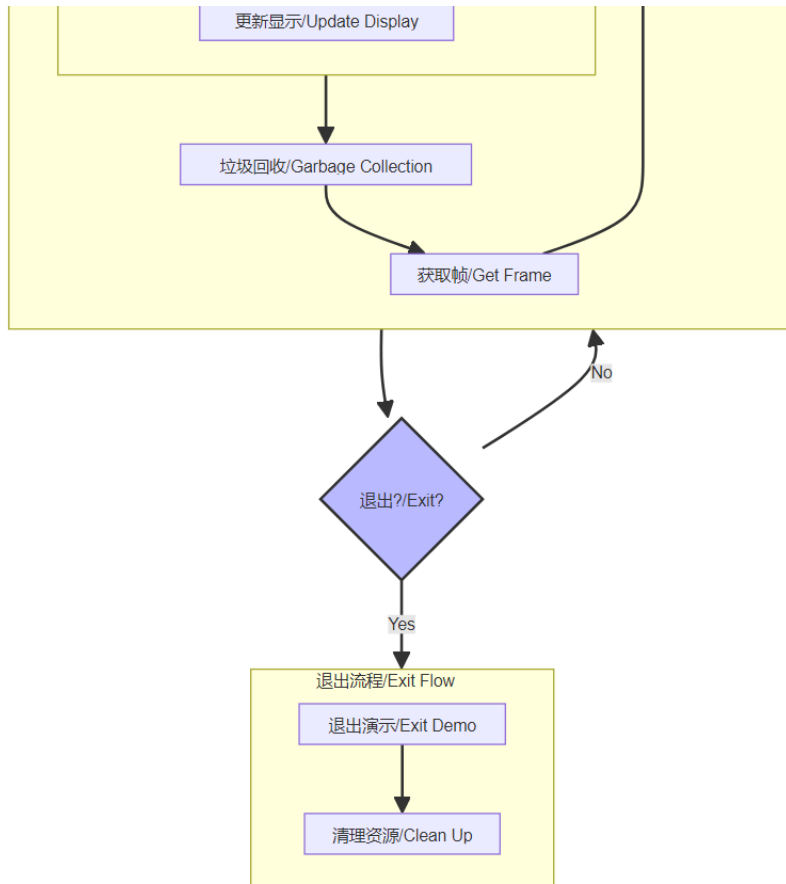
```

# Draw results function
def draw_result(self,pl,dets):
    """
    Draw detection results
    pl: Pipeline instance
    dets: Detection results
    """
    with ScopedTiming("display_draw",self.debug_mode >0):
        if dets:
            pl.osd_img.clear()
            for i in range(len(dets[0])):
                x, y, w, h = map(lambda x: int(round(x, 0)), dets[0][i])
                # Draw rectangle box and label
                pl.osd_img.draw_rectangle(x,y, w, h, color=self.color_four[dets[1]
[i]],thickness=4)
                pl.osd_img.draw_string_advanced( x , y-50,32,
                    " " + self.labels[dets[1][i]] + " " + str(round(dets[2][i],2)) ,
                    color=self.color_four[dets[1][i]])
            else:
                pl.osd_img.clear()

```

The flowchart is as follows:





Algorithm Overview

What is YOLO?

YOLO (You Only Look Once) is a widely popular series of object detection algorithms. Here are the main features:

YOLO Basic Principles:

- Divide the image into a grid
- Directly predict bounding boxes and class probabilities
- Single-stage detection, fast speed

YOLOv8n Features (n stands for nano, the smallest version):

- Faster inference speed
- Smaller model size (~3MB)
- Uses C2f module and SPPF structure
- Suitable for mobile device deployment

Main Performance Comparison:

YOLOv8n on COCO dataset:

- AP: 37.3%
- Speed: ~30% faster than v5

- Parameters: 3.2M